

UNIVERSIDAD AUTÓNOMA DE BAJA CALIFORNIA
FACULTAD DE INGENIERÍA ARQUITECTURA Y DISEÑO

INGENIERÍA EN SOFTWARE Y TECNOLOGÍAS

EMERGENTES



Paradigmas de la programación

T2: Léxico, sintaxis y semántica

ADRIAN BALDERAS ROSAS 373488

Carlos Gallegos

CODIGO EN C

1. Procesamiento léxico

Se tokeniza como:

- `node_t` → identificador (tipo definido por el usuario).
- `*` → operador puntero.
- `crearNodo` → identificador (nombre de función).
- `(` → delimitador de inicio de parámetros.
- `int` → palabra reservada (tipo primitivo).
- `dato` → identificador (parámetro).
- `)` → delimitador de cierre de parámetros.
- `{` → inicio de bloque.

Palabras reservadas: `typedef`, `struct`, `int`, `return`, `if`, `else`, `while`, `void`.

- **Identificadores:** `node_s`, `node_t`, `crearNodo`, `insertarNodo`, `imprimirLista`, `cabeza`, `dato`, `nuevoNodo`, `siguiente`, `actual`, `main`.
- **Constantes:** `1`, `3`, `5`, `0`, `NULL`.
- **Símbolos:** `{}` `()` `[]` `;` `,` `*` `&` `->`.
- **Operadores:** `=`, `==`, `!=`, `*`, `&`, `->`.

2. Procesamiento sintáctico

```
typedef struct node_s {  
    int dato;  
    struct node_s* siguiente;  
} node_t;
```

`typedef` (palabra clave) → declaración de tipo → `struct` anónimo llamado `node_s` → contiene `int dato` y un puntero a `struct node_s`.

```
node_t* crearNodo(int dato) { ... }
```

Cumple la regla → *Declaración de función*: `<tipo> <identificador>(<parámetros>)` { `<bloque>` }.

```
while(actual->siguiente != NULL) {  
    actual = actual->siguiente;  
}
```

estructura válida:

```
while (expresión) { bloque }.
```

3. Procesamiento semántico

```
node_t* nuevoNode = (node_t*)malloc(sizeof(node_t));
```

- malloc devuelve void* → en C requiere conversión explícita a node_t*.
- Se verifica que sizeof(node_t) es válido porque node_t está definido. Correcto semánticamente.

```
if (nuevoNode == NULL)
```

- Compara puntero con NULL → permitido.

```
nuevoNode->dato = dato;
```

- nuevoNode es un node_t*, dato es int.
- ->dato accede a campo de tipo int.
- Tipos compatibles.

```
insertarNode(&cabeza, 1);
```

- cabeza es node_t*.
- &cabeza es node_t**.
- La función espera node_t**. Correcto.

```
imprimirLista(cabeza);
```

- cabeza es node_t*.
- La función espera node_t*. Correcto.

En while(actual != NULL) { ... } dentro de imprimirLista:

- Verificación semántica: se puede recorrer lista mientras puntero no sea nulo. Correcto.

CÓDIGO EN PY

1. Procesamiento léxico

Palabras clave (keywords): class, def, return, if, else, while, is, None, print.

Identificadores: Node, __init__, self, dato, siguiente, insertar_nodo, cabeza, nuevo_nodo, actual, imprimir_lista, main.

Operadores y símbolos: =, ., :, (), ,, ->, ==, is not.

Literales: 1, 3, 5, "HEAD -> ", "NULL".

F-strings: f"{actual.dato} -> " → token especial de formato.

2. Procesamiento sintáctico

```
class Node:
    def __init__(self, dato):
        self.dato = dato
        self.siguiente = None
```

Estructura válida:

class <Nombre>: <bloque> con método __init__ como constructor.

```
def insertar_nodo(cabeza, dato):
    nuevo_nodo = Node(dato)
```

...

Cumple regla de definición de función:

def <nombre>(<parametros>): <bloque>.

```
while actual.siguiente is not None:
```

```
    actual = actual.siguiente
```

Valido: while <condición>: <bloque>.

```
if __name__ == "__main__":
    main()
```

Correcta sintácticamente: if <expresión>: <bloque>.

3. Procesamiento semántico

```
class Node:
```

- Se crea un tipo de dato definido por el usuario.
- Atributos: dato (cualquiera, no tipado en Python) y siguiente (otro Node o None).

nuevo_nodo = Node(dato)

- Crea un objeto de la clase Node.
- dato se asigna correctamente a self.dato.

En insertar_nodo:

- Caso if cabeza is None: retorna el nuevo nodo como cabeza.
- Caso contrario: se recorre la lista (while actual.siguiente is not None).
- Se agrega nuevo_nodo al final.
- Retorna siempre la referencia de la cabeza.

En imprimir_lista:

- Recorre la lista con while actual is not None.
- Imprime datos encadenados hasta NULL.
- Correcto semánticamente.